

A Complete Stack for Efficient Vision Model Deployment on Edge Devices with Z-Ant: optimization of an ONNX deployment library

Matteo Scuratti
Ingegneria Informatica
Politecnico di Milano
Milano, Italia
matteo2.scuratti@mail.polimi.it

Mattia Sicoli
Ingegneria Informatica
Politecnico di Milano
Milano, Italia
mattia.sicoli@mail.polimi.it

Abstract—The deployment of neural networks on micro-controllers and edge devices remains challenging due to stringent memory, compute, and energy constraints. We present optimizations to the Z-Ant pipeline, a Zig-based framework for embedded neural network deployment, focusing on micro-operations (UOps) decomposition and kernel fusion techniques. Our approach decomposes each ONNX graph node into sequences of 30 elementary micro-operations, enabling a greedy fusion engine to merge compatible UOp stacks (e.g., BiasAdd $\rightarrow$ ReLU) into single composite kernels. This eliminates intermediate buffers and call overhead while generating self-contained Zig code with compile-time loop specialization. The optimized pipeline bridges the performance gap between automatically generated and hand-tuned kernels, achieving significant improvements in inference latency and memory efficiency on resource-constrained embedded devices.

Index Terms—neuralnetworks, cnn, edgeai, embedded, zig

I. INTRODUCTION: CURRENT STATE OF THE ART TECHNOLOGIES AND Z-ANT

As of now, the deployment of a network on embedded devices is an inefficient, cumbersome, and unstable endeavor: While training a model has become a linear and accessible practice, the true roadblock is the deployment of the model, most open solutions target deployment on GPUs and other specialized hardware; deployment on embedded devices instead requires development of a tailored kernel written by hand, a task that is both difficult and prone to implementation mistakes, or constricts the user to the usage of a few, and outdated, frameworks that can be divided in two distinct categories:

- Interpreted at runtime, which implies a very high amount of overhead both in runtime duration and memory bandwidth required.
- Evaluated using a static library, requiring less overhead than the interpreted approach but still not being optimized for the required use case enough to be satisfactory.

Clearly then deployment of a network on embedded devices is still a very new and growing field that due to the absence of any well established standards provides lots of freedoms and choice in the methodologies developers pick to further it.

A. Z-Ant and its Objectives

Z-Ant is a lightweight framework that generates custom, highly optimized kernels for ONNX models targeting microcontrollers. Unlike general-purpose inference engines, it does not rely on static libraries of operators but compiles each binary from scratch, adopting a philosophy similar to TinyGrad.

Starting from an ONNX model, Z-Ant performs model parsing, shape analysis, and operator fusion to generate optimized Zig code tailored to the target architecture. This compilation-time approach minimizes memory usage, maximizes performance, and reduces energy consumption—making it ideal for constrained environments.

By avoiding the runtime overhead of frameworks like TensorFlow Lite or ONNX Runtime, Z-Ant targets resource-limited devices like the Raspberry Pi Pico, ARM Cortex-M, STM boards.

The pipeline includes ONNX validation, tensor and memory layout optimization, hardware-aware code generation, and static library compilation. Z-Ant supports over 30 ONNX operators, dynamic input shapes, and multiple architectures including ARM Cortex-M, RISC-V, and x86.

II. LOW HANGING FRUIT OPTIMIZATIONS: MATMUL

Profiling a simple MNIST-8 binary highlighted how most of the runtime was due to the matrix multiplication routines, used by GEMM and convolution.

To establish a consistent testing environment, we design a benchmark that performs matrix multiplication between two matrices: an $N \times (N + \alpha)$ matrix A and an $(N + \beta) \times N$ matrix B , where N is a power of two and α, β are independent random parameters. The test parameters examined are summarized in Table I.

The first optimization effort focused on enabling and fully utilizing the SIMD functionality provided by the tested microcontrollers through Zig's @Vector annotation. The SIMD implementation resulted in a 38% performance improvement compared to the baseline routine when operating on (1024,

TABLE I
TEST PARAMETERS FOR MATRIX MULTIPLICATION BENCHMARK

Data type	N
u8	8
u8	9
u8	10
u16	8
u16	9
u16	10
f32	8
f32	9
f32	10

1024) square matrices. A careful analysis of matrix dimensions reveals three distinct computational scenarios:

- $(M,N) \times (N,H)$ where $M \approx H$
- $(M,N) \times (N,H)$ where $M \gg H$
- $(M,N) \times (N,H)$ where $M \ll H$

Each scenario benefits from a different algorithmic approach. Since the ONNX format specifies matrix dimensions at compile time, the code generation pipeline can automatically select the optimal method for each multiplication operation. While the latter two cases perform optimally with the enhanced SIMD-enabled algorithm, the first case benefits from a cache-aware tiled approach. This implementation partitions the matrix multiplication into smaller blocks of the second matrix, reducing cache misses and maximizing L2 cache utilization. The approach draws inspiration from the optimizations developed by Kazushige Goto in gotoBLAS at Intel [1].

```

1 //Simplified non-simd version on square NxN
2 //To display algorithm coincisely
3 fun blockedmul(
4     comptime T: anytype, N: usize
5     A: [*]T, B: [*]T, C: [*]T,
6     block_size: usize, //L1 cache block size
7 ) void {
8     //For each row of A
9     for(0..N) |r| {
10         //Take a block of B's columns.
11         var b: usize = 0;
12         while (b < N) : (b += block_size) {
13             //Take a chunk of that block.
14             var c: usize = 0;
15             while (c < N) : (c += block_size) {
16                 //For each row of the block
17                 for(0..16) |sc| {
18                     //For each cell of that row
19                     for(0..16) |i| {
20                         //Add partial result
21                         //to C's corresponding cell.
22                         C[r * N + b + i] +=
23                             A[r * N + c + sc] *
24                             B[c * N + sc * N + b + i];
25                     }
26                 }
27             }
28         }
29     }
30 }

```

The complete implementation, extended to support non-square matrices, proves optimal for matrices of size (M,N) where N exceeds the processor's L1 cache block size. When these constraints are not satisfied, the system defaults to the SIMD-enhanced baseline algorithm.

III. LIMITS WITH A FIXED KERNEL APPROACH

The described code generation pipeline exhibited several significant limitations that impacted its overall efficiency:

- Static methods cannot be concatenated, resulting in unavoidable cache misses for operations that could otherwise be fused. Each contiguous group of elements processed by fusible operations incurs additional memory access overhead.
- Each static method generates intermediate tensors with compile-time known dimensions, consuming space in the binary's .bss section and creating substantial RAM overhead.
- Different algorithms optimal for different contexts cannot be efficiently integrated within the static compilation approach.
- While static fused kernels can be implemented, the combinatorial explosion of fusion possibilities across different algorithmic variants renders this approach impractical.

A critical manifestation of these limitations occurred during the YOLO deployment on a Raspberry Pi Pico 2, where the binary size exceeded the device's capacity due to intermediate tensors occupying static memory space that could not be dynamically allocated or deallocated. The cumulative impact of these constraints necessitated a comprehensive redesign of Z-Ant's code generation pipeline from the ground up.

The figure above illustrates the memory footprint as reported by Massif, clearly highlighting the limitations imposed by the current approach, particularly the significant static memory allocation.

IV. PROPOSED SOLUTION: UOPS AND KERNEL FUSION

Here is detailed the new and currently implemented operation lowering pipeline that Z-Ant uses to convert an ONNX model into a runnable binary that can be flashed onto embedded devices.

A. General overview and core idea

Starting from the ONNX file provided by the user, Z-Ant parses the raw file and creates multiple representations that gradually decompose the model into many unit components. The compilation pipeline follows a systematic approach that progressively transforms the neural network model:

- **ONNX parsing and linearization** – The model graph is read and converted into a topologically sorted node list, maintaining shape, datatype, and attribute metadata
- **Pattern matching and fusion** – are detected and merged into single CompositeOps, eliminating intermediate buffers and reducing function call overhead
- **Lowering to UOp IR** – Each high-level operation becomes a sequence of micro-operations including memory

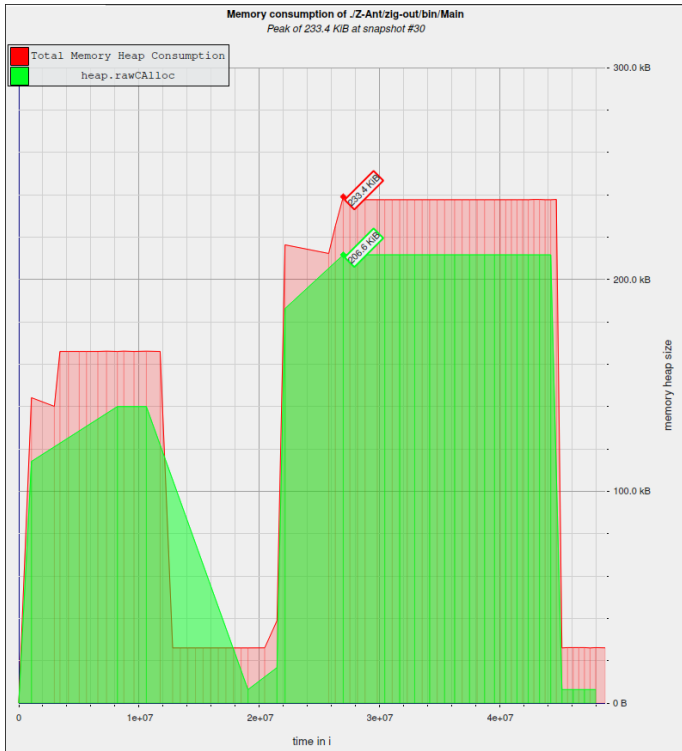


Fig. 1. Massif output showing memory consumption and its limitations.

ops (Load, Store), arithmetic ops (Add, Sub), and scheduling directives (Tile_M, Vectorize)

- **Scheduling and optimization** – Performance markers are inserted and parameters auto-tuned via beam search to match target hardware characteristics
- **Zig code generation** – Multiple Zig-Renderers converts the UOp sequence into idiomatic Zig code, optimizing loops as much as possible using comptime feature of Zig language
- **Compilation and deployment** – The generated code is then compiled into a self-contained executable with an optimized kernel.

B. Parsing

From a given ONNX file, Z-Ant creates an Abstract Syntax Tree (AST) representation of the model which captures the operational structure and execution order of the neural network. This parsing phase serves as the foundation for subsequent optimization and compilation stages.

1) *ONNX to AST Conversion*: The parsing process begins by loading and analyzing the ONNX model file. The ONNX format provides a standardized representation of neural networks through a computational graph structure, where nodes represent operations and edges represent data flow between operations. Z-Ant transforms this graph-based representation into a hierarchical AST that preserves the semantic meaning while enabling efficient traversal and manipulation.

Z-Ant aims to repurpose the parsing techniques first innovated by TinyGrad [2] and adapt them for single core low

powered embedded devices.

During the AST construction phase, each ONNX node is mapped to corresponding Z-Ant internal representations. The parser maintains crucial metadata including:

- Operation types and their parameters
- Input and output tensor specifications
- Data types and tensor shapes
- Control flow dependencies

2) *Pattern Matching and Operation Fusion*: Once the AST is generated, Z-Ant analyzes the tree structure to identify blocks of operations that can be fused together for optimization purposes. This analysis employs pattern matching techniques to recognize common computational patterns that frequently appear in neural network models.

The pattern matching system identifies opportunities for:

- Sequential operations that can be combined into single kernels
- Mathematically equivalent operation sequences
- Memory access patterns that can be optimized
- Redundant computations that can be eliminated

All of these will be found using a beam search algorithm, that will parse through the entire AST for these occurrences. [3]

It should be noted that the complete pattern matching and fusion capabilities will be implemented in successive versions of Z-Ant, with the current implementation focusing on basic operation identification and grouping.

3) *Lowering to Micro-Operations*: After operations are grouped and analyzed, Z-Ant employs specialized lowering functions to decompose high-level ONNX operations into Z-Ant's internal micro-operations (UOPs). This lowering process bridges the gap between the abstract mathematical operations defined in ONNX and the concrete computational primitives that can be efficiently executed on target hardware.

The lowering functions perform several critical transformations:

- **Operation Decomposition**: Complex ONNX operations are broken down into sequences of simpler uops that map more directly to hardware capabilities
- **Memory Layout Optimization**: Tensor layouts are optimized for the target execution environment
- **Type Specialization**: Generic operations are specialized for specific data types and tensor dimensions
- **Hardware Abstraction**: Operations are expressed in terms of hardware-agnostic primitives that can later be mapped to specific accelerator instructions

This multi-stage parsing and lowering approach ensures that Z-Ant can efficiently handle diverse neural network architectures while maintaining the flexibility to target different hardware platforms. The resulting uop representation serves as the intermediate form for subsequent optimization passes and code generation phases.

C. UOps

An UOp (where U should be read as 'micro') is the smallest operation in the lowering pipeline, and contains all the information it needs to then be rendered as merged code.

An UOp is defined by this struct:

```

1  pub const UOp = struct {
2      id: usize,
3      op: UOpType,
4      dtype: DType,
5      src: []const usize,
6      arg: ?Any = null,
7  };

```

- An ID - Unique numerical ID that allows for identifications of individual UOps.
- An OP - Enum tag that marks which operation the struct instance represents.
- A DType - Data type on which the UOp acts.
- Array of sources - Array of IDs that details from which other UOps the arguments on which the UOp should act.
- Any field - A tagged Enum field that allows for any particular UOp to have some special metadata tied to it that it might need to use.

Any UOp is designed to do a specific, irreducible operation that could be called during any subroutine. Any larger operation, such as an element-wise add between two matrices, is built from smaller UOps.

Renderers are the way in which the pipeline turns a stack of individual UOps into compilable Zig code. For brevity and cleanliness, most operations are tied to a specific if not unique renderer struct in charge of turning them, and other similarly structured UOps, into Zig code. They are in charge of validating the contents of a UOp such that no invalid UOp is rendered. (Such as a GEP UOp with no indexes). Each UOp adds an entry to what Z-Ant calls the "PtrMap", a map containing all variable names to which each UOp refers. Each entry is tied to the ID of the UOp that owns that variable, so that the renderer knows that name to embed in the source to correctly refer to the values it acts on.

D. Views

Just with UOps alone, memory management would be cumbersome and inefficient, Z-Ant therefore relies on what it calls 'Views'.

A view is relatively simple in its structure; a view is simply a set of strides that adds information regarding the structure of the underlying memory it's tied to and what it represents, be it a 3x3x3 Tensor or a single-dimensional array of 27 elements. Every view is stored inside a map that ties it to the ID of the UOp that allocated the memory it details; it can be changed or updated at any moment to reflect any change in shape or structure of the tensor it represents, without moving any memory or copying any data, reducing the number of intermediate tensors needed.

Any memory allocated by the UOps is always tied to a view, which is implicitly called by any UOp that attempts to access or modify the data it contains. As of now, the current View functionality is limited and only poised to aid with dimensional index flattening, but is kept generic to allow for greater functionality and added complexity that will be detailed a few paragraphs ahead.

Views are instrumental in reducing the memory needed by the pipeline, as they can be modified in-place, removing the need for any intermediate tensors. As shape is dynamically loaded and changed compared to the previous statically shaped tensor structs, this reduces the memory footprint significantly.

The complete pipeline therefore looks like this, note that the diagram includes the kernel fusion stage, that as of the writing of this report is still not part of the implemented Z-Ant pipeline.

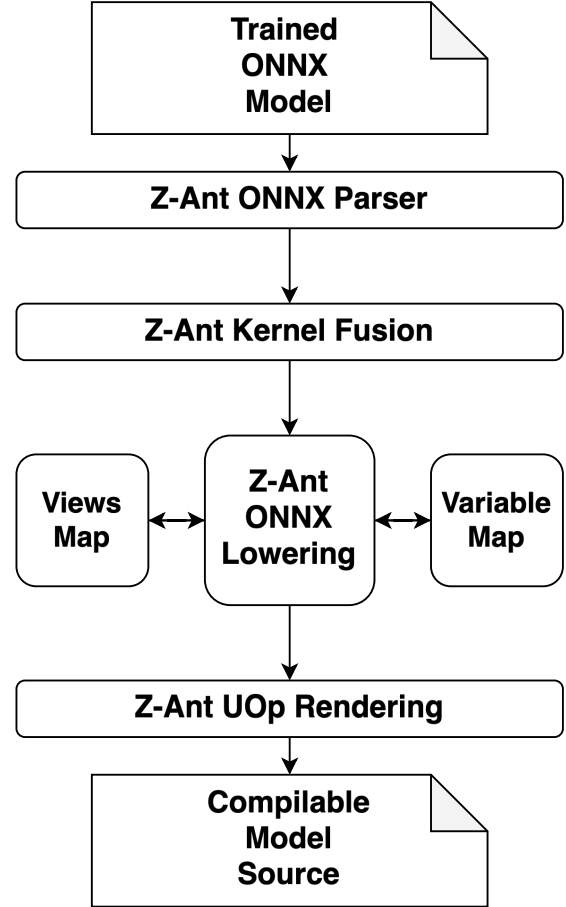


Fig. 2. The complete Z-Ant pipeline

The kernel fusion would be implemented as a greedy graph node merging algorithm that flattens mergeable operations in a single line of code, reducing considerably the amount of double/triple accesses to the same memory location, and therefore reducing the number of cache misses that the previous static approach entailed.

V. THE NEW PIPELINE AT WORK: GENERATING AN OPTIMIZED MATMUL

Once we had developed a working pipeline, we decided to test its effectiveness by implementing a matrix multiplication operation to evaluate the performance improvements achieved through our compilation approach. The MatMul operation serves as an excellent benchmark since it is one of the most

computationally intensive and memory-bound operations in neural network inference.

Our testing focused on comparing the memory usage and execution characteristics of the newly generated MatMul against the previous implementation. The results were measured using Massif, a heap profiler that provides detailed memory usage analysis over time.

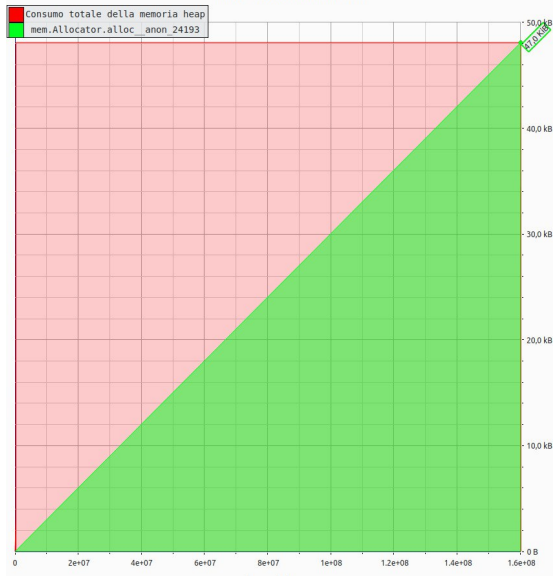


Fig. 3. Memory usage comparison between old and new MatMul implementations measured with Massif

To illustrate the effectiveness of our compilation pipeline, we present an example of the generated MatMul code. The following Zig code snippet demonstrates the matrix multiplication kernel produced by Z-Ant for a 2x2 by 2x3 matrix multiplication:

```

1 pub fn generated_kernel(allocator: std.mem.Allocator, input_0: []const f32,
2   input_1: []const f32) ![]f32 {
3   const output_2 = try allocator.alloc(f32, 6);
4   var idx_3: i32 = 0; // RANGE (uop 3)
5   while (idx_3 < 2) : (idx_3 += 1) {
6     var idx_4: i32 = 0; // RANGE (uop 4)
7     while (idx_4 < 3) : (idx_4 += 1) {
8       var acc_5: f32 = 0;
9       var idx_6: i32 = 0; // RANGE (uop 6)
10      while (idx_6 < 2) : (idx_6 += 1) {
11        // GEP: Get Element Pointer for input matrices
12        const addr_7 = @intFromPtr(input_0_ptr) +
13          ((@as(usize, @intCast(idx_3)) * 2) +
14           (@as(usize, @intCast(idx_4)) * 1))
15          * @sizeOf(f32);
16        const addr_8 = @intFromPtr(input_1_ptr) +
17          ((@as(usize, @intCast(idx_6)) * 3) +
18           (@as(usize, @intCast(idx_4)) * 1)) * @sizeOf(f32);
19        // LOAD operations from computed addresses
20        var buf_9: f32 = @as(*const f32, @ptrFromInt(addr_7)).*;
21        _ = &buf_9;
22        var buf_10: f32 = @as(*const f32, @ptrFromInt(addr_8)).*;
23        _ = &buf_10;
24        acc_5 += buf_9 * buf_10;
25      }
26      // STORE result to output matrix
27      const addr_13 = @intFromPtr(output_2_ptr) +
28        ((@as(usize, @intCast(idx_3)) * 3) +
29         (@as(usize, @intCast(idx_4)) * 1)) * @sizeOf(f32);
30      @as(*f32, @ptrFromInt(addr_13)).* = acc_5;
31    }
32  }
33  return output_2;
34 }

```

Listing 1. Generated MatMul kernel in Zig

The experimental results demonstrated significant memory optimization achievements. We successfully implemented a

functional MatMul operation that consumes approximately 5 times less memory compared to the previous MatMul implementation. This substantial reduction in memory footprint can be attributed to the elimination of intermediate buffers through our pattern matching and fusion stages, as well as the more efficient memory management strategies employed in the generated Zig code.

However, our analysis revealed that we still face challenges in terms of processing time. The current implementation, while memory-efficient, does not yet achieve optimal execution speed. This performance limitation is primarily due to the absence of advanced optimization techniques that we plan to implement in future versions of Z-Ant, specifically the kernel fusion system combined with SIMD (Single Instruction, Multiple Data) operations and cache optimization strategies.

VI. PLANNED DEVELOPEMENTS AND THE FUTURE OF Z-ANT

To further enhance the optimization capabilities of Z-Ant, we plan to implement sophisticated algorithms that can autonomously determine when it is beneficial to employ SIMD instructions and cache optimization techniques based on the specific characteristics of the target platform. This adaptive optimization approach will involve generating multiple code variants for each operation and executing benchmark tests to identify the optimal implementation according to user-defined constraints and performance criteria.

The platform-aware optimization system will analyze hardware specifications such as cache sizes, SIMD instruction set availability, memory bandwidth, and computational throughput to make informed decisions about optimization strategies. For instance, on platforms with limited cache but abundant SIMD capabilities, the system might prioritize vectorization over cache blocking techniques. Conversely, on memory-constrained devices, it might favor cache-friendly algorithms even at the expense of some computational parallelism. [4]

Furthermore, we intend to expand our memory management capabilities by introducing more sophisticated view mechanisms. These advanced views will enable us to extract even greater performance from resource-constrained hardware by implementing intelligent memory reuse patterns, automatic data layout transformations, and dynamic buffer allocation strategies. The enhanced view system will analyze tensor access patterns and automatically reorganize data in memory to minimize cache misses and maximize spatial locality.

The advanced view mechanisms will also incorporate automatic memory coalescing, where multiple small buffers are combined into larger, more cache-friendly allocations, and intelligent prefetching strategies that anticipate data requirements based on the computational graph structure. These "views" will not be limited to contiguous strides of data; instead, they will be able to represent non-contiguous data segments in memory as if they were a single, unified block. These optimizations will be particularly beneficial for edge devices and embedded systems where memory resources are severely limited.

These results establish a solid foundation for further development and exploration of the Z-Ant compilation system. The significant memory improvements achieved demonstrate the viability of our approach, while the identified performance bottlenecks provide clear directions for future enhancements. This positions our work as a promising starting point for developing a more comprehensive and performance-oriented neural network compilation framework that can intelligently adapt to diverse hardware constraints and optimization objectives.

REFERENCES

- [1] K. Goto and R. A. v. d. Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Trans. Math. Softw.*, vol. 34, no. 3, May 2008. [Online]. Available: <https://doi.org/10.1145/1356052.1356053>
- [2] mesozoic-egg [GitHub], “TinyGrad Notes: Pattern Matcher,” https://mesozoic-egg.github.io/tinygrad-notes/20241112_pm.html, November 2024.
- [3] —, “TinyGrad Notes: Beam,” https://mesozoic-egg.github.io/tinygrad-notes/20241203_beam.html, December 2024.
- [4] S. Zheng, R. Chen, M. Li, Z. Ye, L. Ceze, and Y. Liang, “vmcu: Coordinated memory management and kernel optimization for dnn inference on mcus,” in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 452–464. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/d5a655b8b373737b4f2aea8f78e5e754-Paper-Conference.pdf